

RAG CHEAT SHEET



1. WHAT IS RAG?

Retrieval-Augmented Generation (RAG) combines the power of LLMs with external knowledge sources to generate accurate, context-aware answers.

RAG allows LLMs to "know" what they don't know.

2. RAG PIPELINE (HIGH LEVEL)

1. User Query → 2. Retriever (Search) → 3. Relevant Chunks → 4. LLM (Generate) → 5. Final Answer

Retrieve relevant context → Provide to LLM → Generate accurate answer.

3. WHY USE RAG?

- Reduces hallucinations by grounding answers in real data.
- Use private or proprietary data securely.
- Provides up-to-date and domain-specific knowledge.
- No need to retrain or fine-tune LLMs.
- Improves trust, accuracy and explainability.

4. CORE COMPONENTS OF RAG

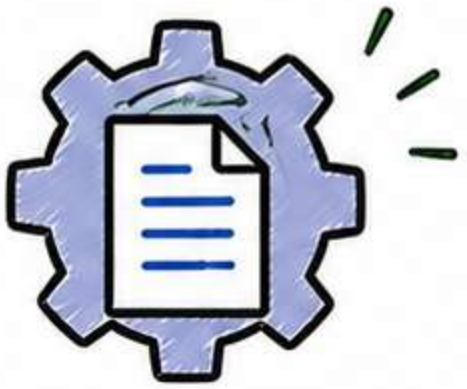
- **Documents** – Raw data from various sources.
- **Embeddings** – Convert text into vectors.
- **Vector Database** – Store and search embeddings efficiently.
- **Retriever** – Finds the most relevant information.
- **LLM** – Generates the final response using retrieved context.

REMEMBER: LLM reasons, RAG provides knowledge.

By Ketan Sagare



RAG CHEAT SHEET



DOCUMENT PROCESSING

1 DATA SOURCES

- PDF Documents
- Word Documents (DOCX)
- Web Pages / HTML
- Databases (SQL / NoSQL)
- APIs / Other Systems

2 PREPROCESSING

- Clean text (remove noise, headers, footers)
- Remove duplicates
- OCR for scanned documents
- Extract metadata (title, author, date, source)
- Normalize text (whitespace, encoding)
- Handle tables, images and lists

3 CHUNKING STRATEGIES

Fixed Size
Split into chunks of fixed number of tokens.

Recursive / Hierarchical
Split by paragraphs, then sentences, then words.

Semantic Chunking
Group semantically similar sentences together.

Parent-Child
Store small chunks + their parent context.

4 METADATA TO CAPTURE

Source	File name, URL, System
Date	Creation or last updated
Author	Document author / owner
Page / Section	Page number or section id
Other Tags	Category, department, keywords, access level

5 BEST PRACTICES

- Choose the right chunk size (200–500 tokens is a good start).
- Keep chunks small enough for precision, large enough for context.
- Preserve important context (use overlap if needed).
- Store rich metadata for better filtering and retrieval.
- Validate chunks manually on a small sample.

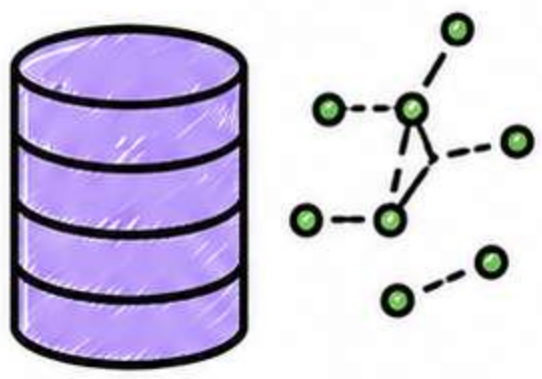
REMEMBER:

- Quality input → Quality retrieval → Quality output.
- Good document processing is the foundation of a strong RAG system.

By Ketan Sagare



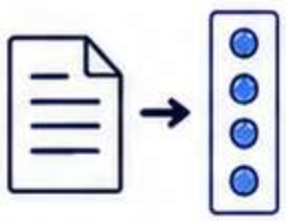
RAG CHEAT SHEET



EMBEDDINGS & VECTOR DATABASES

1 EMBEDDINGS

Convert text into dense vectors so that similar meaning → closer vectors.



`[0.23 -0.11 0.68 ... 0.35]`

Vector Representation

Good Embeddings =

- Capture semantic meaning
- Work across domains
- Improve retrieval quality

2 POPULAR EMBEDDING MODELS

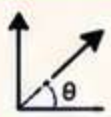
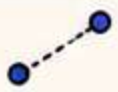
- **OpenAI** (text-embedding-3-large)
  High quality, general purpose
- **BGE (BAAI)**
  Strong performance for retrieval
- **E5 (Alibaba)**
  Optimized for search & retrieval
- **Sentence Transformers**
  Open source, customizable
- **Cohere Embed**
  Good for multilingual use cases

3 VECTOR DATABASES

FAISS	Meta	Fast, open source, in-memory search
Chroma	Open Source	Easy to use, good for dev
Pinecone	Managed	Fully managed, scalable, reliable
Milvus	Open Source	Highly scalable, enterprise ready
Weaviate	Open Source	GraphQL support, rich metadata

4 SIMILARITY SEARCH

Find vectors most similar to the query vector.

Cosine Similarity 	$sim(A, B) = \frac{A \cdot B}{\|A\| \, \|B\|}$	Most Common
Dot Product 	$sim(A, B) = A \cdot B$	Good for normalized vectors
Euclidean Distance 	$dist(A, B) = \sqrt{\sum (A_i - B_i)^2}$	Less Common

5 KEY POINTS

- Embeddings turn text into numbers that capture meaning.
- Vector DB stores embeddings and allows fast similarity search.
- Choice of embedding model + vector DB impacts retrieval quality.
- Use metadata with vectors for better filtering.
- Higher dimension ≠ always better. Quality matters more!



REMEMBER: Better Embeddings + Right Vector DB + Good Similarity Search = Better Retrieval → Better Answers

By Ketan Sagare

RAG CHEAT SHEET

RETRIEVAL OPTIMIZATION

1 RETRIEVAL METHODS

Dense Retrieval
Uses vector embeddings to find semantically similar chunks.

Sparse Retrieval
Uses keywords / BM25 to match exact or partial terms.

Hybrid Retrieval
Combines dense + sparse results for better recall and precision.

2 IMPROVE RETRIEVAL QUALITY

Query Expansion
Expand query using LLM or synonyms to get more relevant results.

Re-Ranking
Re-rank top-k results using a cross-encoder for better relevance.

Metadata Filtering
Filter by source, date, author, category to narrow down results.

Top-K Tuning
Adjust top-k value based on task and context window size.

3 RETRIEVAL PARAMETERS

`top_k`	Number of chunks to retrieve from vector DB.
`score_threshold`	Minimum similarity score to consider a chunk.
`mmr`	Maximal Marginal Relevance to reduce redundancy.
`fetch_k`	Fetch more candidates before re-ranking.
`hybrid_weight`	Weightage between dense and sparse scores.

4 RETRIEVAL METRICS

- **Recall@K** — How many relevant chunks are retrieved.
- **Precision@K** — How many retrieved chunks are relevant.
- **MRR** — Quality of ranking. Higher is better.
- **nDCG** — Considers ranking position of relevant chunks.

Ideal Ranking

1 ✓
2 ✓
3 ✗
4 ✓
5 ✗

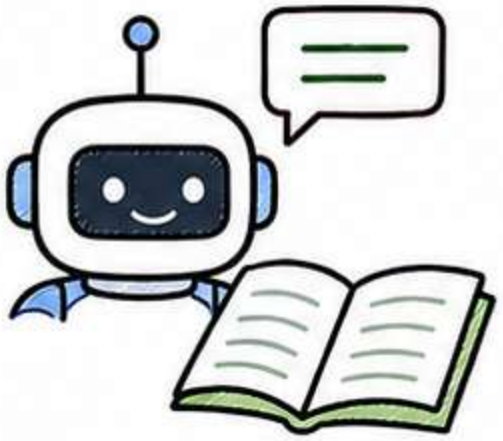
Relevant
Not Relevant

5 COMMON RETRIEVAL PROBLEMS

Irrelevant Results	→ Improve embeddings, use re-ranking, better chunking, add metadata filters.
Low Recall	→ Query expansion, hybrid search, increase top_k.
Redundant Chunks	→ Use MMR, better chunking strategy.
Contect Overflow	→ Reduce chunk size, select top relevant, summarize context.

REMEMBER: Good Retrieval = Relevant Context = Better Answers

By Ketan Sagare



RAG CHEAT SHEET



PRODUCTION RAG & BEST PRACTICES

1 EVALUATION METRICS

Faithfulness
Answer is factually consistent with context.

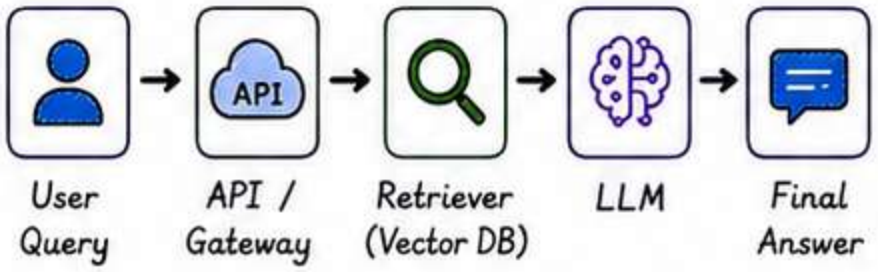
Groundedness
Answer is supported by retrieved context.

Relevance
Answer directly addresses the user query.

Context Precision
Retrieved context contains only necessary information.

Context Recall
Retrieved context covers all needed information.

2 PRODUCTION ARCHITECTURE



- Add caching at multiple layers (query, embeddings, retrieval results).
- Use async / streaming for better latency.
- Implement fallbacks for empty or low-confidence retrievals.
- Log everything for monitoring and evaluation.

3 MONITORING & OBSERVABILITY

Latency
Track end-to-end and component wise latency.

Cost
Monitor token usage and API / model costs.

Hallucination Rate
Track rate of ungrounded or incorrect answers.

Cache Hit Rate
Measure cache effectiveness (retrieval + response).

Token Usage
Track input, output and total tokens.

4 BEST PRACTICES

- Use Hybrid Search (Dense + Sparse) for better recall.
- Re-rank results before sending to LLM.
- Keep chunk size optimal (200–500 tokens) with overlap.
- Add metadata and use filters to improve precision.
- Write clear system prompts and answer guidelines.
- Validate with evaluations and user feedback.
- Continuously update data and re-index.



5 COMMON PITFALLS TO AVOID

- Using only dense search without considering sparse search.
- Too large chunks → low precision, context overflow.
- Low top_k → misses relevant information.
- No re-ranking → poor quality context to LLM.
- Ignoring metadata → irrelevant or noisy results.
- No evaluation → don't know if system is improving.

REMEMBER:
Great RAG is NOT just retrieval + generation.
It's about the right data, right context, right answer.

By Ketan Sagare